

Document number: P1679R3

Date: 2020-06-13

Project: WG21, Library Working Group

Authors: Wim Leflere wim.leflere@gmail.com, Paul Fee paul.f.fee@gmail.com

string contains function

1. Abstract

This paper proposes to add member function `contains` to class templates `basic_string` and `basic_string_view`. This function checks, whether or not a string contains a given substring.

2. History

2.1. R3

Small wording update based on LWG feedback

Rephrased case sensitivity section

Feature test macro added

2.2. R2

Small wording update based on LWG feedback

2.3. R1

Wording added

Made all functions constexpr, after `std::string` was made constexpr [1](#)

Merged content from P1657R0 [2](#)

2.4. R0

Initial version

3. Motivation

Checking, whether or not a given string contains a given substring is a common task, that is missing from the standard library.

3.1. Other (standard) libraries

Standard libraries of many other programming languages include routines for performing such a check, for example:

- Python: operator `in`, which calls an object's `__contains__(self, item)` method [3](#)
- Java: class `String` `contains` method [4](#)
- C#: class `String` `Contains` method [5](#)
- Rust: class `string` `contains` method [6](#)

And so on.

Also, some C++ libraries (other than the standard library) that implement a string type include such methods. For example, Qt library has classes `QString` [7](#) and `QStringRef` (analogous to `std::string_view`) which have `contains` member functions.

C++ will be easier to teach to people coming from other languages as they may already be familiar with the `contains` method in the other language's string class.

3.2. Existing substring checks

A range of options exist for substring checking.

```
std::string haystack = "no place for needles";
```

Using the C library:

```
if (strstr(haystack.c_str(), "needle"))
```

Using the C++ standard library:

```
if (haystack.find("needle") != std::string::npos)
```

Using Boost algorithms library [8](#):

```
if (boost::contains(haystack, "needle"))
```

The proposed changes would provide a concise, unambiguous method for substring checking in which the intent is clearly expressed.

```
if (haystack.contains("needle"))
```

3.3. Why not `find != npos`?

The 'standard' [9](#) way of checking if a string contains a substring is to use the `find` member function.

```
if (str.find(substr) != std::string::npos)
    std::cout << "found!\n";
```

But using `find` requires that one extra step of thinking when writing it.

You're trying to do something positive (check if contains) but you have to do something negative (check inequality).

And one extra step when reading the code.

Are we looking for the actual position? Or checking if the string contains a substring? Or checking if the string doesn't contain a substring?

A `contains` member function would make the intention of the programmer more clear and make the code more readable.

```
if (str.contains(substr))  
    std::cout << "found!\n";
```

The proposed change would improve teachability of C++ for beginners as the `contains` function better matches the intention of the programmer. And because it is a simpler construct to write and remember than using `find`.

3.4. Three string checking Musketeers

The string `contains` function would complete the three string checking musketeers, together with the string prefix and suffix check, `starts_with` and `ends_with` [10](#).

4. Design considerations

4.1 Standard library vs core language change

Python uses the `in` operator, such as:

```
if 'needle' in haystack:
```

Adopting a similar approach in C++ would involve a new keyword. A new keyword risks breaking backwards compatibility with code already using `in` for other purposes, such as variable names. Hence changes to the standard library are preferred.

4.2. Member function vs free function

This proposal adds member function `contains` to class templates `basic_string` and `basic_string_view`. Another option considered was to add a free function `contains` to namespace `std`, as in Boost [8](#). The drawback of a free function is that the order of parameters of a free function is ambiguous, `contains(string, substring)` vs `contains(substring, string)`.

A member function offers consistency with other popular languages, such as Java, C# and Rust. It's also consistent with `starts_with` and `ends_with` [10](#).

4.3 `string::contains` vs `map::contains`

Containers such as `set` and `map` have a `contains` method. For `multiset` and `multimap` containers, this offers a performance boost over the `count` method since `contains` can return on the first match. With `set` and `map`, the benefits are API consistency with `multiset` and `multimap` along with clearer expression of intent by returning a `bool` rather than a count.

```
std::set<foo> haystack;
if (haystack.count(needle)) { /* found */ }
if (haystack.contains(needle)) { /* clearly found */ }
```

The proposed `contains` method for substring checks is not directly analogous to the container operation. Rather than search for a member within the container, a `contains` operation on a string means to search for a substring.

Since the proposed method is being called on a string (or `string_view`) object, the context is clear. The same method name reuse can be seen with the `find` method provided by both containers and string objects. Likewise, the Python `in` operator performs substring searches on string and membership searches on containers.

4.4. Case sensitivity

The `starts_with`, `ends_with` and `find` methods are case sensitive. Likewise the proposed `contains` member function is also case sensitive.

Some libraries offer case insensitive searches.

For example, Boost string algorithms provides `icontains` [11](#).

Qt's `QString::contains` takes a parameter that defaults to case sensitive, but allows case insensitivity to be specified.

However, case sensitivity is a complex topic for character sets beyond ASCII. Therefore the scope of this proposal is limited to case sensitive substring checks.

4.5 Overload set

The `starts_with` and `ends_with` methods each have three overloads. This proposal has the same set of overloads:

```
// basic_string:
constexpr bool contains(basic_string_view<charT, traits> str) const noexcept;
constexpr bool contains(charT ch) const noexcept;
constexpr bool contains(const charT* str) const;

// basic_string_view:
constexpr bool contains(basic_string_view<charT, traits> str) const noexcept;
constexpr bool contains(charT ch) const noexcept;
constexpr bool contains(const charT* str) const;
```

An overload accepting a `basic_string` is not required since `basic_string` has a non-explicit conversion operator to `basic_string_view`.

5. Wording

5.1. `basic_string`

In `[basic.string]`, add:

```
constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;  
constexpr bool contains(charT x) const noexcept;  
constexpr bool contains(const charT* x) const;
```

After `[string.ends.with]`, add:

```
basic_string::contains [string.contains]  
  
constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;  
constexpr bool contains(charT x) const noexcept;  
constexpr bool contains(const charT* x) const;
```

Effects: Equivalent to:

```
return basic_string_view<charT, traits>(data(), size()).contains(x);
```

5.2. `basic_string_view`

In `[string.view.template]`, add:

```
constexpr bool contains(basic_string_view x) const noexcept;  
constexpr bool contains(charT x) const noexcept;  
constexpr bool contains(const charT* x) const;
```

In `[string.view.ops]`, add:

```
constexpr bool contains(basic_string_view x) const noexcept;  
constexpr bool contains(charT x) const noexcept;  
constexpr bool contains(const charT* x) const;
```

Effects: Equivalent to: `return find(x) != npos;`

5.3. Feature test macro

In `[version.syn]`, add:

```
#define __cpp_lib_string_contains YYYYMMML // also in <string>, <string_view>
```

Adjust the placeholder value as needed so as to denote this proposal's date of adoption.

6. References

1. WG21 P0980R0 - Making `std::string constexpr` - <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0980r0.pdf>
2. WG21 P1657R0 - String substring checking - <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1657r0.pdf>
3. Python3 Language Reference - Expressions, Membership test operations - <https://docs.python.org/3/reference/expressions.html#in>
4. Java™ Standard Edition 10 API - Class String - [https://docs.oracle.com/javase/10/docs/api/java/lang/String.html#contains\(java.lang.CharSequence\)](https://docs.oracle.com/javase/10/docs/api/java/lang/String.html#contains(java.lang.CharSequence))
5. .NET Core 2.2 API - String.Contains Method - <https://docs.microsoft.com/en-us/dotnet/api/system.string.contains?view=netcore-2.2>
6. Rust 1.0 API - Struct `std::string::String` - <https://doc.rust-lang.org/std/string/struct.String.html#method.contains>
7. Qt 5.12 documentation - QString Class - <https://doc.qt.io/qt-5/qstring.html#contains>
8. Boost 1.70.0 documentation - `boost::algorithm::contains` function - https://www.boost.org/doc/libs/1_70_0/doc/html/boost/algorithm/contains.html
9. StackOverflow - C++ string contains - <https://stackoverflow.com/questions/2340281/check-if-a-string-contains-a-string-in-c/2340309#2340309>
10. WG21 P0457R2 - String Prefix and Suffix Checking - <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0457r2>
11. Boost 1.70.0 documentation - `boost::algorithm::icontains` function - https://www.boost.org/doc/libs/1_70_0/doc/html/boost/algorithm/icontains.html